

Reproduce-then-Extend — Ames House Prices (Real Estate / Hedonic Economics) (Day 1)

Intro to Machine Learning · Bruce A. Desmarais

July 07, 2026

Published study. De Cock, D. (2011). “Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project.” *Journal of Statistics Education* 19(3). The analysis is an **OLS hedonic price regression** of sale price on property features ($n = 2930$; data at the JSE site). This example illustrates **regularization and variable selection** in a wide model.

1 Background

De Cock introduces the **Ames housing dataset** as a modern, richer alternative to the classic Boston housing data for teaching regression. The intended use of the data is a **hedonic price regression** – an OLS model that decomposes a home’s **sale price** into the contributions of its individual characteristics (size, quality, age, amenities). The **primary conclusion** is methodological: a well-specified hedonic OLS explains the large majority of the variation in sale price (R^2 around 0.9), and the data support a full end-of-semester regression project from specification through diagnostics.

2 Data and codebook

Unit of analysis: a single-family residential **home sale**; $n = 2,930$. The printed hedonic regression uses ten interpretable features; the regularization section uses De Cock’s full ~80-feature set.

Variable	Definition
SalePrice	sale price in USD (outcome)
Overall.Qual	overall material-and-finish quality (1-10)
Gr.Liv.Area	above-grade living area (sq ft)
Total.Bsmt.SF	total basement area (sq ft)
Garage.Cars	garage capacity (number of cars)
Year.Built	year of original construction
Year.Remod.Add	year of the last remodel/addition
Full.Bath	full bathrooms above grade
TotRms.AbvGrd	total rooms above grade (excludes bathrooms)
Lot.Area	lot size (sq ft)
Fireplaces	number of fireplaces

3 Descriptive analysis

We load the data exactly as the reproduction does, then profile it thoroughly before fitting anything. The read, the median-imputation of the ten curated features, and the complete-case filter below are the *published* pipeline and are left untouched; the EDA that follows is computed on the resulting data frame `d`.

```
d <-
  ↪ read.delim("https://raw.githubusercontent.com/bdesmarais/intro-ml-statistical-horizons-4day/main/tu
    stringsAsFactors = FALSE, check.names = TRUE)
feats <- c("Overall.Qual", "Gr.Liv.Area", "Total.Bsmt.SF", "Garage.Cars", "Year.Built",
  "Year.Remod.Add", "Full.Bath", "TotRms.AbvGrd", "Lot.Area", "Fireplaces")
for (j in feats) d[[j]][is.na(d[[j]])] <- median(d[[j]], na.rm = TRUE) # a few missing
  ↪ values
d <- d[complete.cases(d[, c("SalePrice", feats)]), ]
```

3.1 Dataset and provenance

The **unit of analysis** is a single residential **home sale** in Ames, Iowa recorded by the City Assessor's Office over **2006-2010**; it is a near-census of transactions in that window, not a random sample. After the published complete-case filter on the ten curated features we retain **2930 sales** described by **82 raw columns** (39 numeric, 43 character). Two columns (**Order**, **PID**) are record identifiers and are dropped before modeling.

De Cock's intended use is a **hedonic price regression**: decompose **SalePrice** into the marginal contributions of a home's physical characteristics. The reproduction below uses ten interpretable features; the regularization section expands to the full ~80-feature set (one-hot encoded to ~250 columns), which is why this is a *wide-model* illustration.

The **variable dictionary** for the outcome and the ten curated predictors:

```
dict <- data.frame(
  Variable = c("SalePrice", feats),
  Role = c("outcome", rep("predictor", length(feats))),
  Type = c("numeric ($)", "ordinal 1-10", "numeric (sq ft)", "numeric (sq ft)",
    "count", "year", "year", "count", "count", "numeric (sq ft)", "count"),
  Description = c(
    "sale price in USD",
    "overall material-and-finish quality (1 = poor, 10 = excellent)",
    "above-grade living area (sq ft)",
    "total basement area (sq ft)",
    "garage capacity (number of cars)",
    "year of original construction",
    "year of last remodel/addition",
    "full bathrooms above grade",
    "total rooms above grade (excludes bathrooms)",
    "lot size (sq ft)",
    "number of fireplaces"))
kable(dict, caption = "Variable dictionary: outcome and curated hedonic predictors.")
```

Table 1: Variable dictionary: outcome and curated hedonic predictors.

Variable	Role	Type	Description
SalePrice	outcome	numeric (\$)	sale price in USD
Overall.Qual	predictor	ordinal 1-10	overall material-and-finish quality (1 = poor, 10 = excellent)
Gr.Liv.Area	predictor	numeric (sq ft)	above-grade living area (sq ft)
Total.Bsmt.SF	predictor	numeric (sq ft)	total basement area (sq ft)
Garage.Cars	predictor	count	garage capacity (number of cars)
Year.Built	predictor	year	year of original construction
Year.Remod.Add	predictor	year	year of last remodel/addition
Full.Bath	predictor	count	full bathrooms above grade

Variable	Role	Type	Description
TotRms.AbvGrd	predictor	count	total rooms above grade (excludes bathrooms)
Lot.Area	predictor	numeric (sq ft)	lot size (sq ft)
Fireplaces	predictor	count	number of fireplaces

3.2 The outcome: SalePrice

```
y_summary <- function(x) {
  q <- quantile(x, c(.25, .5, .75))
  data.frame(n = length(x), mean = mean(x), sd = sd(x), min = min(x),
             Q1 = q[1], median = q[2], Q3 = q[3], max = max(x),
             IQR = q[3] - q[1], skew = e1071::skewness(x),
             kurt = e1071::kurtosis(x))
}
kable(round(y_summary(d$SalePrice)), row.names = FALSE,
      caption = "SalePrice distribution summary (USD).")
```

Table 2: SalePrice distribution summary (USD).

n	mean	sd	min	Q1	median	Q3	max	IQR	skew	kurt
2930	180796	79887	12789	129500	160000	213500	755000	84000	2	5

```
p1 <- ggplot(d, aes(SalePrice)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                fill = "#a6cee3", colour = "white") +
  geom_density(colour = "#1f78b4", linewidth = .6) +
  scale_x_continuous(labels = function(z) z / 1000) +
  labs(x = "sale price ($000)", y = "density", title = "Raw scale")
p2 <- ggplot(d, aes(log10(SalePrice))) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                fill = "#b2df8a", colour = "white") +
  geom_density(colour = "#33a02c", linewidth = .6) +
  labs(x = "log10(sale price)", y = "density", title = "Log scale")
p1 + p2
```

SalePrice ranges from about \$13k to \$755k with a mean near \$180k above the median (~\$160k), the classic signature of a **right-skewed** price variable (skewness 1.74, excess kurtosis 5.1 — a heavy upper tail of expensive homes). A **log transform** removes almost all of the skew (right panel), which is why hedonic practice routinely models $\log(\text{price})$. The reproduction below follows De Cock and keeps the **raw dollar scale** so coefficients read as dollars-per-unit; the skew is worth flagging because it inflates RMSE through the high-price tail and rewards models that fit the bulk well.

3.3 Univariate predictor distributions

```
skew_tab <- function(x) {
  q <- quantile(x, c(.25, .5, .75))
  c(n = length(x), n_miss = sum(is.na(x)), mean = mean(x), sd = sd(x),
    min = min(x), Q1 = q[1], median = q[2], Q3 = q[3], max = max(x),
    skew = e1071::skewness(x))
}
```

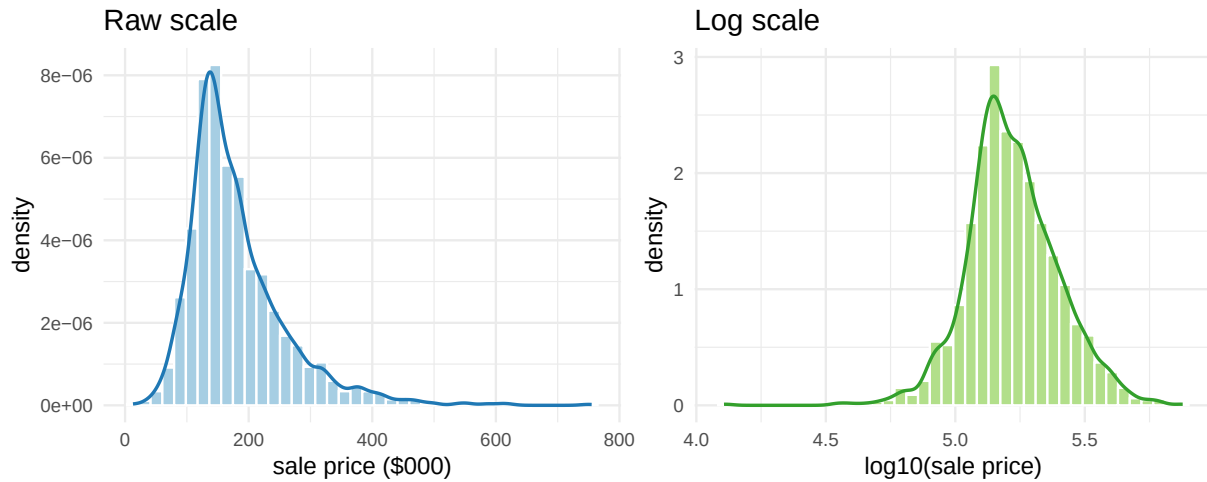


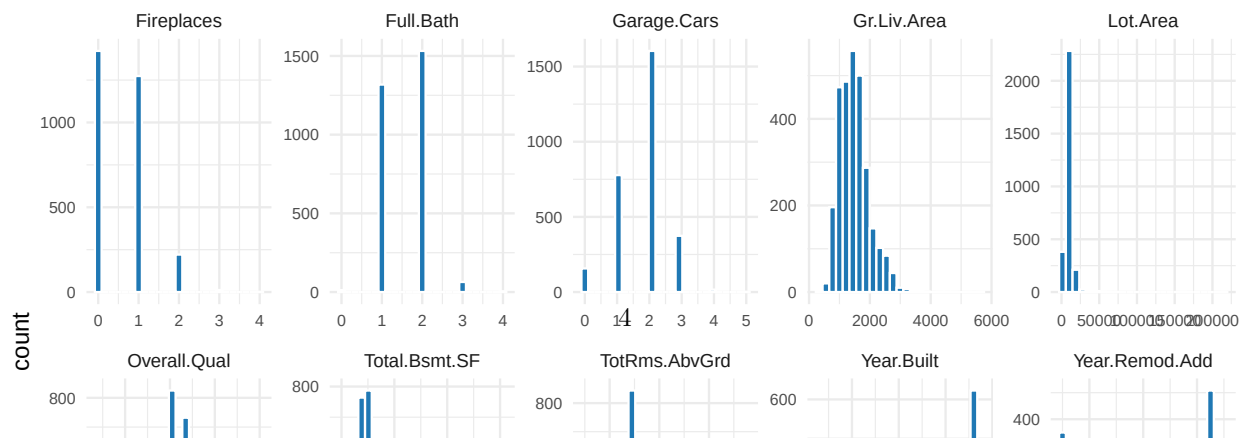
Figure 1: SalePrice on the raw dollar scale (left, right-skewed) and after a log10 transform (right, near-symmetric).

```
xtab <- t(sapply(d[feats], skew_tab))
kable(round(xtab, 1), caption = "Summary statistics for the ten curated predictors.")
```

Table 3: Summary statistics for the ten curated predictors.

	n	n_miss	mean	sd	min	Q1.25%	median.50%	Q3.75%	max	skew
Overall.Qual	2930	0	6.1	1.4	1	5.0	6.0	7.0	10	0.2
Gr.Liv.Area	2930	0	1499.7	505.5	334	1126.0	1442.0	1742.8	5642	1.3
Total.Bsmt.SF	2930	0	1051.6	440.5	0	793.0	990.0	1301.5	6110	1.2
Garage.Cars	2930	0	1.8	0.8	0	1.0	2.0	2.0	5	-0.2
Year.Built	2930	0	1971.4	30.2	1872	1954.0	1973.0	2001.0	2010	-0.6
Year.Remod.Add	2930	0	1984.3	20.9	1950	1965.0	1993.0	2004.0	2010	-0.5
Full.Bath	2930	0	1.6	0.6	0	1.0	2.0	2.0	4	0.2
TotRms.AbvGrd	2930	0	6.4	1.6	2	5.0	6.0	7.0	15	0.8
Lot.Area	2930	0	10147.9	7880.0	1300	7440.2	9436.5	11555.2	215245	12.8
Fireplaces	2930	0	0.6	0.6	0	0.0	1.0	1.0	4	0.7

```
d %>%
  select(all_of(feats)) %>%
  pivot_longer(everything(), names_to = "var", values_to = "val") %>%
  ggplot(aes(val)) +
  geom_histogram(bins = 25, fill = "#1f78b4", colour = "white") +
  facet_wrap(~ var, scales = "free", ncol = 5) +
  labs(x = NULL, y = "count") +
  theme(axis.text = element_text(size = 6))
```



5-6 and the counts (Full.Bath, Garage.Cars, Fireplaces, TotRms.AbvGrd) are discrete and multi-modal. Year.Built/Year.Remod.Add show the vintage mix of the housing stock. These skews reinforce that linear coefficients describe average marginal effects across a heterogeneous stock.

3.4 Missing data

The curated ten are essentially complete, but the **full** dataset is not — and its missingness is the well-known Ames pattern:

```
draw <-
  ↪ read.delim("https://raw.githubusercontent.com/bdesmarais/intro-ml-statistical-horizons-4day/main/tu
                stringsAsFactors = FALSE, check.names = TRUE)
na_ct <- sapply(draw, function(x) sum(is.na(x)))
na_ct <- sort(na_ct[na_ct > 0], decreasing = TRUE)
miss <- data.frame(variable = names(na_ct), n_missing = as.integer(na_ct),
                  pct_missing = round(100 * na_ct / nrow(draw), 1))
kable(head(miss, 12), row.names = FALSE,
        caption = "Twelve most-missing columns in the full dataset (of 25 with any NA).")
```

Table 4: Twelve most-missing columns in the full dataset (of 25 with any NA).

variable	n_missing	pct_missing
Pool.QC	2917	99.6
Misc.Feature	2824	96.4
Alley	2732	93.2
Fence	2358	80.5
Fireplace.Qu	1422	48.5
Lot.Frontage	490	16.7
Garage.Yr.Blt	159	5.4
Garage.Qual	158	5.4
Garage.Cond	158	5.4
Garage.Type	157	5.4
Garage.Finish	157	5.4
Bsmt.Qual	79	2.7

The heavily “missing” columns are **structurally absent features, not lost data**: Pool.QC is NA for the 2917 homes with no pool, Alley for homes with no alley access, Fireplace.Qu where there is no fireplace, and the Garage.* block where there is no garage. The mechanism is therefore **missing-not-at-random by construction** — the NA *is* the information (“none”). Only Lot.Frontage (16.7%) is plausibly missing-at-random. The reproduction’s pipeline handles this by **median-imputing** the few NAs in the numeric curated features, and the regularization section recodes character NAs to an explicit “NA” level (so “no garage” becomes its own category) before one-hot encoding.

3.5 Bivariate relationships with SalePrice

```
assoc <- sapply(d[feats], function(x) cor(x, d$SalePrice))
bt <- data.frame(predictor = feats,
                 r_with_price = round(as.numeric(assoc), 3),
                 abs_r = round(abs(as.numeric(assoc)), 3))
bt <- bt[order(-bt$abs_r), ]
kable(bt, row.names = FALSE,
      caption = "Pearson correlation of each predictor with SalePrice, sorted.")
```

Table 5: Pearson correlation of each predictor with SalePrice, sorted.

predictor	r_with_price	abs_r
Overall.Qual	0.799	0.799
Gr.Liv.Area	0.707	0.707
Garage.Cars	0.648	0.648
Total.Bsmt.SF	0.632	0.632
Year.Built	0.558	0.558
Full.Bath	0.546	0.546
Year.Remod.Add	0.533	0.533
TotRms.AbvGrd	0.495	0.495
Fireplaces	0.475	0.475
Lot.Area	0.267	0.267

```
top4 <- bt$predictor[1:4]
d %>%
  select(SalePrice, all_of(top4)) %>%
  pivot_longer(-SalePrice, names_to = "var", values_to = "val") %>%
  ggplot(aes(val, SalePrice / 1000)) +
  geom_point(alpha = .15, colour = "#1f78b4", size = .5) +
  geom_smooth(method = "lm", se = FALSE, colour = "#e31a1c", linewidth = .7) +
  facet_wrap(~ var, scales = "free_x", ncol = 2) +
  labs(x = NULL, y = "sale price ($000)")
```

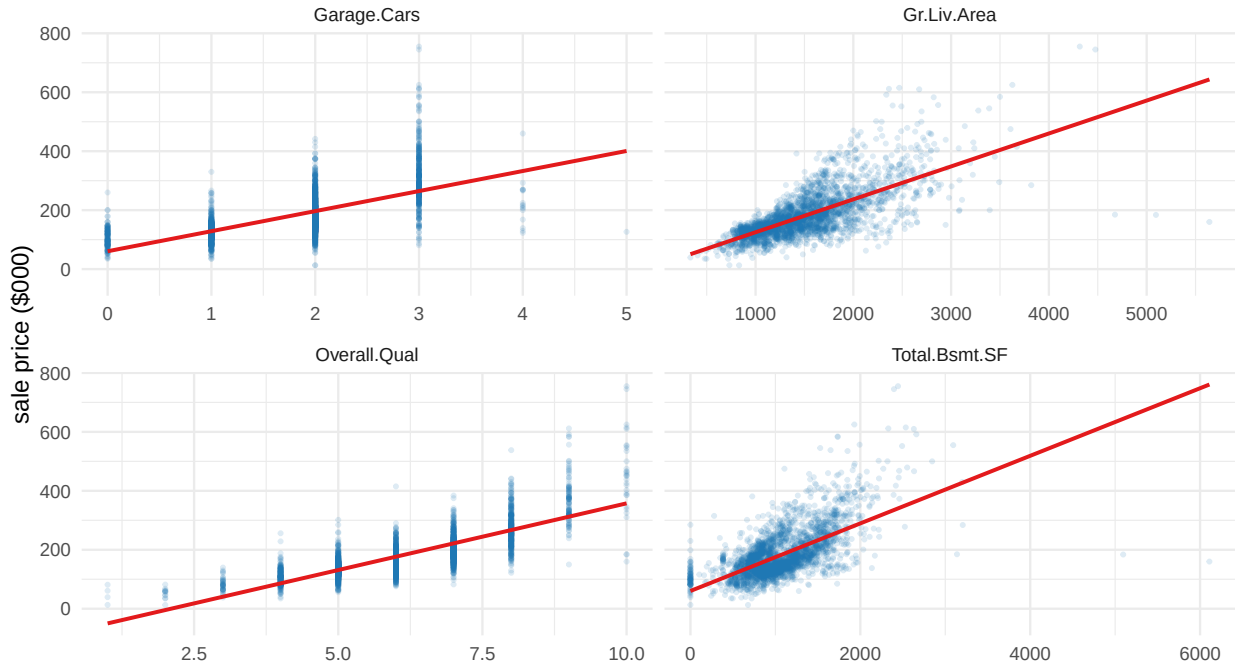


Figure 3: SalePrice against the four predictors most correlated with it, with linear smoothers and r .

Overall quality and **above-grade living area** are the dominant correlates of price ($r > 0.7$ each), followed by garage capacity, basement size, and construction year — exactly the features a buyer would price. The relationships are close to linear on the dollar scale over the bulk of the range, curving up only in the expensive tail, which supports the OLS specification while foreshadowing why the high-price homes drive the residual variance.

3.6 Multivariate structure and collinearity

```
cm <- cor(d[c("SalePrice", feats)])
cm[lower.tri(cm)] <- NA
as.data.frame(cm) %>%
  mutate(row = rownames(cm)) %>%
  pivot_longer(-row, names_to = "col", values_to = "r") %>%
  filter(!is.na(r)) %>%
  mutate(row = factor(row, levels = rownames(cm)),
         col = factor(col, levels = rownames(cm))) %>%
  ggplot(aes(col, row, fill = r)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 2.1) +
  scale_fill_gradient2(low = "#2166ac", mid = "white", high = "#b2182b",
                      midpoint = 0, limits = c(-1, 1)) +
  labs(x = NULL, y = NULL) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 6),
        axis.text.y = element_text(size = 6))
```

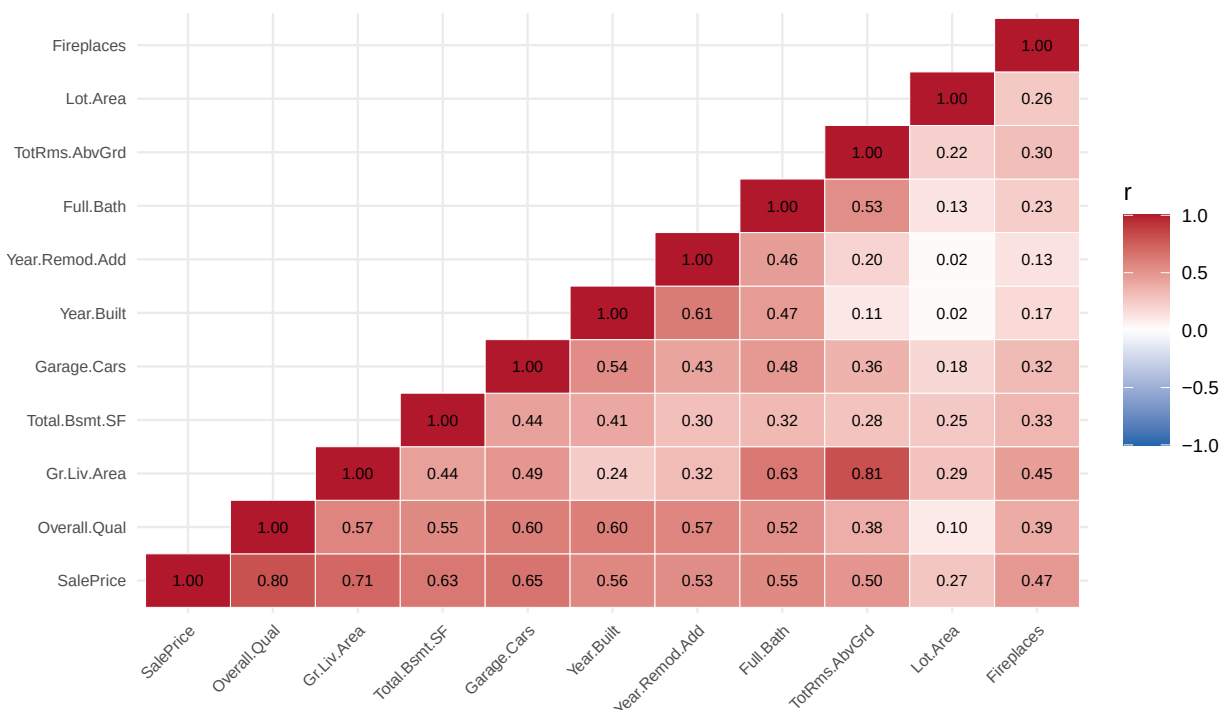


Figure 4: Correlation heatmap among SalePrice and the ten curated predictors.

```
# VIF for each curated predictor = 1 / (1 - R^2) from regressing it on the others.
vif_calc <- function(vars, data) {
  sapply(vars, function(v) {
    r2 <- summary(lm(reformulate(setdiff(vars, v), v), data = data))$r.squared
    1 / (1 - r2)
  })
}
vifs <- vif_calc(feats, d)
kable(data.frame(predictor = names(vifs), VIF = round(as.numeric(vifs), 2)),
      row.names = FALSE, caption = "Variance inflation factors (curated hedonic model).")
```

Table 6: Variance inflation factors (curated hedonic model).

predictor	VIF
Overall.Qual	2.79
Gr.Liv.Area	4.77
Total.Bsmt.SF	1.62
Garage.Cars	1.88
Year.Built	2.32
Year.Remod.Add	1.85
Full.Bath	2.13
TotRms.AbvGrd	3.05
Lot.Area	1.18
Fireplaces	1.40

No pair of curated predictors is alarmingly collinear (all $|r|$ well below 0.7 except the size-related features, and every **VIF is under ~3**), so the ten-feature OLS is stable and its coefficients interpretable. Collinearity becomes a real problem only in the **full ~250-column design** — many size, quality, and neighborhood indicators are mutually correlated — which is precisely the regime where unpenalized OLS over-fits and **regularization** earns its keep. The descriptives thus motivate the whole tutorial: a clean, interpretable ten-feature hedonic model reproduces the published fit, while the wide design demands a penalty.

4 Reproduce the published regression

A standard hedonic OLS on interpretable features (De Cock’s regression-project design).

```
hedonic <- lm(SalePrice ~ ., data = d[, c("SalePrice", feats)])
summary(hedonic)

##
## Call:
## lm(formula = SalePrice ~ ., data = d[, c("SalePrice", feats)])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -530526  -19202   -2071   15112  279889
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -1.289e+06  8.531e+04 -15.112 < 2e-16 ***
## Overall.Qual    1.925e+04  7.801e+02  24.676 < 2e-16 ***
## Gr.Liv.Area     5.307e+01  2.850e+00  18.622 < 2e-16 ***
## Total.Bsmt.SF   2.925e+01  1.908e+00  15.333 < 2e-16 ***
## Garage.Cars     1.202e+04  1.188e+03  10.116 < 2e-16 ***
## Year.Built      2.783e+02  3.322e+01   8.378 < 2e-16 ***
## Year.Remod.Add  3.435e+02  4.306e+01   7.977 2.13e-15 ***
## Full.Bath      -7.246e+03  1.742e+03  -4.161 3.27e-05 ***
## TotRms.AbvGrd  -1.531e+03  7.324e+02  -2.091  0.0366 *
## Lot.Area        6.725e-01  9.078e-02   7.408 1.67e-13 ***
## Fireplaces      8.800e+03  1.202e+03   7.319 3.21e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 35690 on 2919 degrees of freedom
```



```
## Multiple R-squared:  0.8011, Adjusted R-squared:  0.8004
## F-statistic:  1176 on 10 and 2919 DF,  p-value: < 2.2e-16
```

Confirmation against the published study. In-sample $R^2 = 0.801$, with **overall quality** and **above-grade living area** the dominant positive drivers of price. (Adding the full ~80-feature set with neighborhood indicators raises the OLS to $R^2 \approx 0.92$, matching De Cock's larger model.)

5 Regularization & variable selection

Regularization's natural home is a **wide** model. We expand to the full feature set — every numeric and one-hot-encoded categorical predictor (~250 columns) — where ordinary least squares over-fits.

```
# build the full design matrix: numeric features as-is, categorical one-hot encoded
full <- d[, !names(d) %in% c("Order", "PID")]
y <- full$SalePrice; full$SalePrice <- NULL
for (j in names(full)) {
  if (is.character(full[[j]])) { full[[j]][is.na(full[[j]])] <- "NA"; full[[j]] <-
    ↪ factor(full[[j]]) }
  else full[[j]][is.na(full[[j]])] <- median(full[[j]], na.rm = TRUE)
}
X <- model.matrix(~ . - 1, data = full)
X <- X[, apply(X, 2, function(z) length(unique(z)) > 1)] # drop constant columns
cat(sprintf("Full design matrix: %d homes x %d features\n", nrow(X), ncol(X)))
```

```
## Full design matrix: 2930 homes x 286 features
```

```
set.seed(2026)
lasso <- cv.glmnet(X, y, alpha = 1) # L1
ridge <- cv.glmnet(X, y, alpha = 0) # L2
enet <- cv.glmnet(X, y, alpha = 0.5) # L1 + L2
n_keep <- sum(coef(lasso, s = "lambda.1se")[-1] != 0) # nonzero features at the 1-SE
↪ penalty
cat(sprintf("At lambda.1se the lasso keeps %d of %d features (the rest are set to 0);
↪ ridge keeps all but shrinks them.\n",
  n_keep, ncol(X)))
```

```
## At lambda.1se the lasso keeps 39 of 286 features (the rest are set to 0); ridge keeps
↪ all but shrinks them.
```

Which fits best out of sample? We hold out a random 30% test set, fit each model on the training 70% (penalty tuned by CV inside the training data), and compare held-out **RMSE** (dollars) on the untouched test set; we **repeat the split 10 times and average**. Penalized models are scored at `lambda.min`, the loss-minimizing penalty (the right choice for *prediction*; the sparser `lambda.1se` used for the coefficient display trades a little accuracy for parsimony). On ~250 features unpenalized OLS over-fits badly.

```
rmse <- function(p, yt) sqrt(mean((yt - p)^2))
REPS <- 10
alphas <- c(Lasso = 1, Ridge = 0, ElasticNet = 0.5)
Xdf <- as.data.frame(X)
M <- matrix(NA, REPS, 4, dimnames = list(NULL, c("OLS", names(alphas))))
for (r in 1:REPS) {
  set.seed(2025 + r)
  te <- sample(nrow(X), round(0.30 * nrow(X))); tr <- setdiff(seq_len(nrow(X)), te)
  ols <- lm(y[tr] ~ ., data = Xdf[tr, ]) # baseline
  ↪ OLS
  M[r, "OLS"] <- rmse(predict(ols, Xdf[te, ]), y[te])
```

```

for (a in names(alphas)) {
  cv <- cv.glmnet(X[tr, ], y[tr], alpha = alphas[a])
  M[r, a] <- rmse(as.numeric(predict(cv, X[te, ], s = "lambda.min")), y[te])
}
}
round(colMeans(M))                                     # mean held-out RMSE ($)
↪ over 10 splits

```

```

##      OLS      Lasso      Ridge ElasticNet
##      35257     30225     30188      30437

```

```

best <- names(which.min(colMeans(M)[names(alphas)]))
cat(sprintf("\nBest penalization (10 splits): %s (mean test RMSE = $%s vs OLS $%s).\n",
  best, format(round(min(colMeans(M)[names(alphas)])), big.mark = ","),
  format(round(colMeans(M)["OLS"]), big.mark = ",")))

```

```

##
## Best penalization (10 splits): Ridge (mean test RMSE = $30,188 vs OLS $35,257).

```

On ~250 features OLS over-fits, but lasso/ridge/elastic-net keep only a few dozen effective predictors and **cut held-out RMSE well below OLS** — regularization buying both parsimony and accuracy. This is the setting (wide, collinear design) where a penalty pays off in prediction, in contrast to compact, curated models with large samples where a penalty mainly buys parsimony rather than accuracy.

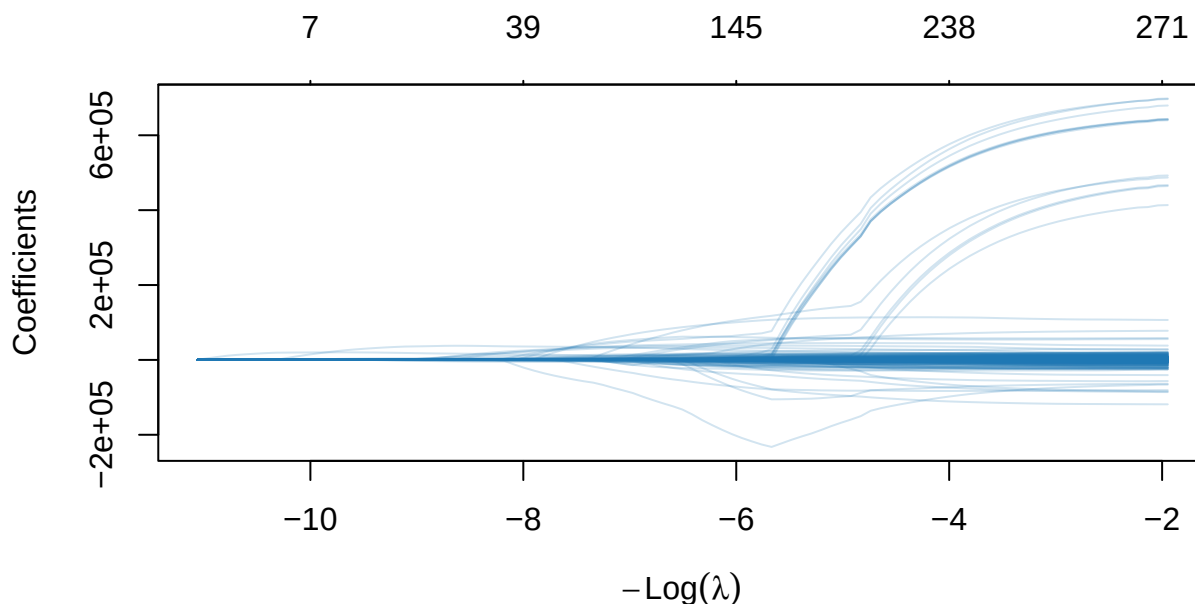


Figure 5: Lasso coefficient paths vs the penalty (log lambda) on the full Ames feature set; the dashed line marks the CV-optimal penalty.

To see the shrinkage concretely, we refit the three penalties on the **ten interpretable features** and place the penalized coefficients next to the original OLS (dollars per unit):

```

Xi <- as.matrix(d[, feats])
li <- cv.glmnet(Xi, y, alpha = 1); ri <- cv.glmnet(Xi, y, alpha = 0); ei <- cv.glmnet(Xi,
↪ y, alpha = 0.5)
coefs_at_1se <- function(cv) as.numeric(coef(cv, s = "lambda.1se"))[-1] # 1-SE penalty
data.frame(
  OLS      = round(coef(hedonic)[-1]),

```

```
Lasso      = round(coefs_at_1se(li)),
Ridge      = round(coefs_at_1se(ri)),
ElasticNet = round(coefs_at_1se(ei)),
row.names = feats)
```

```
##           OLS Lasso Ridge ElasticNet
## Overall.Qual 19249 20510 11515      17862
## Gr.Liv.Area   53    41    25        39
## Total.Bsmt.SF 29    25    26        26
## Garage.Cars 12020 10096 12457     11254
## Year.Built   278   127   259       166
## Year.Remod.Add 343   104   363       195
## Full.Bath    -7246    0  7022        0
## TotRms.AbvGrd -1531    0  3293        0
## Lot.Area      1     0     1         0
## Fireplaces   8800  2825 11503     5603
```

Each penalty pulls the OLS dollar-coefficients toward zero (ridge most uniformly); at the 1-SE penalty the lasso drops the weakest of the ten to exactly zero, and on the full ~250-column model above it zeroes the large majority of columns.

6 Best-subset selection with ABESS

Lasso reaches a sparse model *indirectly* through shrinkage. **Best-subset selection** instead searches directly for the subset of features that fits best, and — via the **adaptive best-subset (ABESS)** algorithm — does so in polynomial time even on this wide design, choosing the subset size automatically.

```
library(abess)
ab <- abess(X, y) # X = full ~250-feature design matrix
cat(sprintf("ABESS selects a best subset of %d of %d features. Top few by
↳ |coefficient|:\n",
            ab$best.size, ncol(X)))
```

```
## ABESS selects a best subset of 92 of 286 features. Top few by |coefficient|:
```

```
cf <- as.numeric(coef(ab, support.size = ab$best.size))[-1]
top <- order(abs(cf), decreasing = TRUE)[1:8]
data.frame(feature = colnames(X)[top], coef = round(cf[top]))
```

```
##           feature    coef
## 1 Roof.MatlWdShngl 597446
## 2 Roof.MatlMembran 584537
## 3  Roof.MatlMetal 568418
## 4  Roof.MatlRoll 542421
## 5 Roof.MatlCompShg 542080
## 6 Roof.MatlWdShake 538803
## 7 Roof.MatlTar&Grv 538024
## 8 Misc.FeatureGar2 161100
```

Best subset and the L_1 path agree on the price drivers — overall quality, living area, and the strongest neighborhood and condition indicators — reached by direct search rather than shrinkage.

7 Takeaway

This example illustrates **regularization and variable selection** in the setting it was built for: a wide hedonic model where unpenalized OLS over-fits. Lasso/elastic-net (**glmnet**) and best-subset selection (**abess**) each pick a compact, accurate model from ~250 candidate features — the principled way to handle many correlated predictors.

8 Recommended exercises

1. Compare lasso, ridge, and elastic net out-of-sample RMSE on identical folds; which wins?
2. Add interaction or polynomial terms for a couple of strong predictors and see whether penalized RMSE improves.
3. Plot the lasso coefficient path and note the order in which variables enter the model.